# Day 4 — Advanced Java Programming Concepts

## Answers, Programs, and Sample Outputs

**Quick theory answers**
**Arrays:** Fixed-size storage for multiple values of the same type; indexes start at 0.
**Strings:** Text objects; Strings are immutable.
**Collections Framework:** Java interfaces and classes for working with groups of objects, such as List, Set, and Map.
**ArrayList:** A resizable ordered list that allows duplicates.
**HashMap:** Stores key-value pairs; keys are unique.
**Exception handling:** Use try/catch to handle errors appropriately.
**File handling:** Read and write files using Java file APIs.
**Streams:** Process data with operations such as filter, map, and sorted.

### Question 1 — Store, search, and sort student records

The program stores students in an ArrayList, searches by ID and name, sorts by name or marks, and catches invalid numeric input.

```
import java.util.*;

class Student {
    private int id;
    private String name;
    private double marks;

    Student(int id, String name, double marks) {
        this.id = id;
        this.name = name;
        this.marks = marks;
    }
    int getId() { return id; }
    String getName() { return name; }
    double getMarks() { return marks; }
    void display() {
        System.out.println(id + " | " + name + " | " + marks);
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        ArrayList<Student> list = new ArrayList<>();
        list.add(new Student(101, "Asha", 86.5));
        list.add(new Student(102, "Rahul", 74));
        list.add(new Student(103, "Neha", 92));

        try {
            System.out.println("All students:");
            for (Student s : list) s.display();

            System.out.print("Search ID: ");
            int id = Integer.parseInt(sc.nextLine());
            boolean found = false;
            for (Student s : list) {
                if (s.getId() == id) {
                    s.display(); found = true; break;
                }
            }
            if (!found) System.out.println("ID not found.");

            System.out.print("Search name: ");
            String name = sc.nextLine();
            found = false;
            for (Student s : list) {
                if (s.getName().equalsIgnoreCase(name)) {
                    s.display(); found = true;
                }
            }
            if (!found) System.out.println("Name not found.");

            System.out.println("Sort: 1 Name, 2 Marks (high to low)");
            int choice = Integer.parseInt(sc.nextLine());
            if (choice == 1)
                list.sort(Comparator.comparing(Student::getName));
            else if (choice == 2)
                list.sort(Comparator.comparingDouble(Student::getMarks).reversed());
            else
                System.out.println("Invalid choice; list order unchanged.");

            System.out.println("Final list:");
            for (Student s : list) s.display();
        } catch (NumberFormatException e) {
            System.out.println("Invalid number entered.");
        } finally {
            sc.close();
        }
    }
}
```

**Sample output (example inputs: ID 102, name Neha, sort choice 2)**

```
All students:
101 | Asha | 86.5
102 | Rahul | 74.0
103 | Neha | 92.0
Search ID: 102
102 | Rahul | 74.0
Search name: Neha
```

```
103 | Neha | 92.0
Sort: 1 Name, 2 Marks (high to low)
2
Final list:
103 | Neha | 92.0
101 | Asha | 86.5
102 | Rahul | 74.0
```

## Question 2 — ArrayList and HashMap

```
import java.util.ArrayList;
import java.util.HashMap;

public class Main {
    public static void main(String[] args) {
        ArrayList<String> names = new ArrayList<>();
        names.add("Asha");
        names.add("Rahul");
        names.add("Neha");
        System.out.println("Names: " + names);

        HashMap<Integer, String> employees = new HashMap<>();
        employees.put(101, "Asha");
        employees.put(102, "Rahul");
        employees.put(103, "Neha");

        System.out.println("Employee 102: " + employees.get(102));
        System.out.println("Has ID 105? " + employees.containsKey(105));
    }
}
```

**Output**

```
Names: [Asha, Rahul, Neha]
Employee 102: Rahul
Has ID 105? false
```

## Question 3 — Strings and Java Streams

```
import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

public class Main {
    public static void main(String[] args) {
        String text = "Java Programming";
        System.out.println(text.length());
        System.out.println(text.toUpperCase());

        List<Integer> marks = Arrays.asList(65, 82, 91, 54, 76);
        List<Integer> passed = marks.stream()
                .filter(mark -> mark >= 70)
                .sorted()
                .collect(Collectors.toList());
        System.out.println(passed);
    }
}
```

**Output**

```
16
JAVA PROGRAMMING
[76, 82, 91]
```

## Question 4 — Write and read a file

```
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;

public class Main {
    public static void main(String[] args) {
        Path path = Path.of("students.txt");
        try {
            Files.writeString(path, "101,Asha,86.5\n102,Rahul,74.0\n");
            System.out.println(Files.readString(path));
        } catch (IOException e) {
            System.out.println("File error: " + e.getMessage());
        }
    }
}
```

**Output**

```
101,Asha,86.5
102,Rahul,74.0
```

## Question 5 — Example of exception handling

```
public class Main {
    public static void main(String[] args) {
        try {
            int number = Integer.parseInt("abc");
            System.out.println(number);
        } catch (NumberFormatException e) {
            System.out.println("Invalid number!");
        }
    }
}
```

**Output**

```
Invalid number!
```

## Short explanations

**Array:** Stores multiple values of the same type with a fixed size.
**String:** Stores text and is immutable.
**ArrayList:** Resizable list that allows duplicates.
**HashMap:** Stores values using unique keys.
**Exception handling:** Handles errors with try/catch.
**File handling:** Reads and writes files.
**Streams:** Processes data using filter, map, and sorted.

## How to run

Run each program separately because every example uses a public class named Main. Save as Main.java, compile with javac Main.java, and run with java Main. The file example creates students.txt in the working directory. Sample output for Question 1 assumes the example inputs shown above.